Computación Concurrente, Paralela y Distribuida, Práctica 4

Estudio del rendimiento y la escalabilidad, instrucciones SIMD

Versión: 3 de febrero de 2026

Contexto

Esta práctica sigue con el estudio de los programas paralelos, pero más que en técnicas para paralelizar, nos centraremos en la *medición de las mejoras*. Dicha **medición** es **clave** para poder saber si un programa que hemos paralelizado obtiene un **rendimiento suficientemente bueno** respecto a su versión secuencial, y lo que es más importante, si seguirá obteniendo ese rendimiento a medida que sigamos aumentando los **recursos** (e.g., cores y memoria). Este último concepto es el de la *escalabilidad*, y es fundamental en la arquitectura de sistemas informáticos de procesamiento de datos en general, y en el paralelismo en particular.

Para poder realizar este estudio usaremos un **programa** que implementa un procesamiento sencillo de unos datos. Más concretamente, aplica diversas operaciones a dos conjuntos de números, que darán como resultado final otro conjunto de números. Este programa lo **ejecutaremos con diversos grados de paralelismo** (e.g., 2x, 4x, 16x) e iremos analizando tanto sus **tiempos** como su **consumo de recursos**.

No obstante, este programa usará **dos tipos de paralelismo**. Por un lado, el programa se paralelizará usando procesos para repartir la carga de trabajo, es decir, se trocearán los conjuntos de números a procesar y se repartirán entre los diversos procesos (técnica ya conocida de *Pooling*). Pero además, por otro lado también se usará paralelismo de bajo nivel disponible en el procesador al usar las *instrucciones vectoriales SIMD* para realizar las operaciones entre los conjuntos de números. Estos dos tipos de paralelismo se estudiarán de forma aislada y combinada en esta práctica.

Objetivos

En esta práctica, mediante un código y diversas formas de ejecutarlo, tanto en local como en la Máquina Remota, tendremos los siguientes objetivos:

- Estudiar y aprender a *cuantificar mejoras del rendimiento* entre varios programas o configuraciones, ya sea de una versión paralela respecto a una versión secuencial, o entre dos versiones paralelas que usan distintas estrategias. Para este análisis usaremos la *Ley de Amdahl*.
- Para un mismo programa o estrategia de paralelización, analizar la evolución de su mejora respecto al grado de paralelismo que usa, es decir, su *estudiar su escalabilidad*. Para este análisis usaremos principalmente la *Ley de Gustafson*.
- Explorar el **rendimiento** que proporciona el uso de *instrucciones vectoriales SIMD* como técnica de paralelización de bajo nivel disponible en el hardware actual de los procesadores.

Índice

1 El programa, explicaciones previas del código **3**
- 1.1 Paralelización . . . 3
- 1.2 Listas y vectores . . . 4
- 1.3 Validación . . . 5
- 1.4 Gestión de la memoria . . . 6
- 1.5 Tareas para la comprensión del código . . . 6

2 Estudio del rendimiento (Amdahl) **8**
- 2.1 Medición simple . . . 8
- 2.2 Medición avanzada . . . 10

3 Estudio de la Escalabilidad (Gustafson) **14**

Índice de Tareas

Tarea 1: Estudio previo del código 6
Tarea 2: Ejecución del código, listas y vectores, secuencial y paralelo 6
Tarea 3: Calculando aceleración con Amdahl (local) 9
Tarea 4: Calculando aceleración con Amdahl (remoto) 9
Tarea 5: Evolución de las aceleraciones, medición en la MR 10
Tarea 6: Estudio de la escalabilidad con Ley de Amdahl y ejecuciones previas 11
Tarea 7: Estudio de la escalabilidad con Ley de Gustafson y ejecuciones previas 14

Antes de empezar (I): Es importante mencionar que los resultados de experimentación de esta práctica y de prácticas pasadas, son todos a modo de ejemplo y para uso académico. Esto es debido fundamentalmente a que haremos análisis de resultados de ejecuciones usando solo 1 medición. En un escenario 'real' en el que se analizase el rendimiento de un programa, sería necesario hacer un estudio algo más extensivo que el actual. Esto se haría principalmente con mediciones provenientes de varias ejecuciones (e.g., las medias), descartando mediciones anómalas (outliers).

Antes de empezar (II): Para las ejecuciones de esta práctica configura la memoria de tu contenedor según las especificaciones de tu equipo tal y como se indica a continuación:

- 8 GiB o menos → Asigna la mitad.
- 12 GiB o más → Asigna 8 GiB.
- En cualquier caso, si en algún momento la ejecución de los códigos de esta práctica es abortada, podría deberse a un fallo por memoria insuficiente.

1. El programa, explicaciones previas del código

Considerando la complejidad técnica del programa usado en esta práctica, así como su configuración y flexibilidad a la hora de usar varios tipos de paralelismo y de estructuras de datos, a continuación se ofrecen explicaciones detalladas de sus diferentes partes.

1.1. Paralelización

La forma de repartir la carga en este código es similar a la Práctica 3, se define un *Pool* de procesos que llevarán a cabo el procesamiento, así como un conjunto de tareas representadas por los trozos de las listas o arrays de números. No obstante, por simplicidad, en este caso el número de tareas y el de procesos será el mismo, de tal forma que cada proceso recibirá solo una tarea. Esto es debido a que todas las tareas requerirán el mismo tiempo de procesamiento (algo que sabemos ya de antemano), lo que implica que la solución óptima es la que usa el mínimo número de tareas, minimizando así la gestión del Pool y las comunicaciones.

Puedes observar en las siguientes líneas cómo se ha troceado el conjunto de datos creando segmentos, que luego se han enviado a los procesos del Pool. Los resultados son después recogidos y fusionados (función '*flatten_list_of_**') para devolver una sola lista o array resultado.

Función de troceo de datos y creación de tareas (codigo1.py)

```
def get_chunks(n, n_proc):
    chunksize = n // n_proc
    proc_chunks = []
    for i_proc in range(n_proc):
        chunkstart = i_proc * chunksize
        if i_proc < n_proc - 1:
            chunkend = (i_proc + 1) * chunksize
        else:
            chunkend = n
        proc_chunks.append((chunkstart, chunkend))
    return proc_chunks
```

Función que reparte las tareas y aglutina resultados (codigo1.py)

```
def process_parallel(n, x1, x2, n_proc=2, funct=process_chunk_list):
    proc_chunks = get_chunks(n, n_proc)
    with Pool(processes=n_proc) as pool:
        data_chunks = list()
        for chunk in proc_chunks:
            ini, end = chunk
            data_chunks.append(((0, end - ini), x1[ini:end], x2[ini:end]))

        proc_results = [pool.apply_async(funct, args=chunk) for chunk in data_chunks]

        # blocks until all results are fetched
        result_chunks = [r.get() for r in proc_results]
        if funct == process_chunk_list:
            return flatten_list_of_lists(result_chunks)
        if funct == process_chunk_array:
            return flatten_list_of_arrays(result_chunks)
```

1.2. Listas y vectores

Este programa realiza cálculos con conjuntos de números, troceando el conjunto a procesar en partes que son luego distribuidas a los procesos siguiendo la técnica de paralelismo de *Pooling*. Sin embargo, hay 2 versiones en el código para procesar estos datos según la estructura usada para almacenarlos.

Por un lado se pueden usar listas tradicionales de Python, con su forma tradicional de aplicar operaciones a listas. Esto es o bien con un bucle 'for', o con una 'list comprehension'. En las siguientes líneas puedes ver la función que realiza las operaciones con listas, usando en este caso 'list comprehension', a fin de tener un código más manejable y breve (el rendimiento es muy similar al bucle for).

Procesamiento de un trozo de lista (codigo1.py)

```
def process_chunk_list(chunk, a, b):
    ini, end = chunk
    result_final = list()

    result1 = [x * y for x, y in zip(a[ini:end], b[ini:end])]  # OP 1
    number_mask = [7] * len(result1)  # OP 2

    for i in range(0, N_ITERS):
        result1 = [x * y for x, y in zip(a[ini:end], b[ini:end])]  # OP 3
        result2 = [x + y for x, y in zip(a[ini:end], b[ini:end])]  # OP 4
        result3 = [x / y for x, y in zip(a[ini:end], b[ini:end])]  # OP 5
        result4 = [x // y for x, y in zip(a[ini:end], b[ini:end])]  # OP 6
        result_final = result1 + result2  # OP 7
        result_final = result_final + result3  # OP 8
        result_final = result_final + result4  # OP 9
        mask = [multiplication_is_even(x, y) for x, y in zip(result_final, number_mask)]  # OP 10
        result_final = [make_even_if_not(x, y) for x, y in zip(result_final, mask)]  # OP 11
    return result_final
```

Por otro lado, es posible realizar las mismas operaciones pero utilizando otra estructura de datos, un array o vector, particularmente uno implementado por NumPy, así como operaciones vectorizables. Observa en las siguientes líneas como se aplican las mismas operaciones que con listas, pero usando funciones vectorizadas y arrays. Las operaciones que se observan comentadas (e.g., *#OP 3*) son llevadas a cabo por NumPy. Será esta librería la que, de forma transparente al programador, si detecta que el procesador dispone de instrucciones vectoriales (muy probable en procesadores modernos), las use para acelerar el procesamiento.

Procesamiento de un trozo de array (codigo1.py)

```
def process_chunk_array(chunk, ax, bx):
    ini, end = chunk
    vec_fun_mult = np.vectorize(multiplication_is_even)
    vec_fun_even = np.vectorize(make_even_if_not)

    result_final = ax[ini:end] * bx[ini:end]  # OP 1
    number_mask = np.ones(len(result_final)) * 7  # OP 2

    for i in range(0, N_ITERS):
        result1 = ax[ini:end] * bx[ini:end]  # OP 3
        result2 = ax[ini:end] + bx[ini:end]  # OP 4
        result3 = ax[ini:end] / bx[ini:end]  # OP 5
        result4 = ax[ini:end] // bx[ini:end]  # OP 6
        result_final = result1 + result2  # OP 7
        result_final = result_final + result3  # OP 8
        result_final = result_final + result4  # OP 9
        mask = vec_fun_mult(result_final, number_mask)  # OP 10
        result_final = vec_fun_even(result_final, mask)  # OP 11
    return result_final
```

Uso de las SIMD en Python: La librería NumPy es capaz de realizar la búsqueda de las instrucciones SIMD disponibles en el procesador, incluso en tiempo real antes de iniciar la ejecución de las operaciones pedidas. Si no las encontrase o no fuera capaz de usarlas, probablemente haría un 'fallback' a un procesamiento clásico mediante un bucle 'for'. Este comportamiento no obstante es debido a que Python es considerado un lenguaje de *alto nivel* y no se espera que el programador use dichas instrucciones de bajo nivel. No obstante, en otros lenguajes como C, o incluso ensamblador, sí que se dejaría esta labor al programador.

1.3. Validación

Recordando la importancia de validar los resultados, también se ha usado una función que recibe listas o arrays de números y realiza varias comprobaciones. Primero se realiza una comprobación simple y superficial de que las longitudes de las listas o vectores coincide. Si no fuera el caso, los resultados no serían ya iguales sin importar el contenido. A continuación se hace una comprobación ya exhaustiva de que para todas las posiciones el resultado coincide (i.e., mismo contenido y en mismo orden, los resultados son iguales). Puedes ver esta función en las siguientes líneas:

Función de validación (codigo1.py)

```
def validate(*args):
    length = len(args[0])
    for res in args[1:]:
        if len(res) != length:
            return False

    length = len(args[0])
    for i in range(0, length):
        value = args[0][i]
        for res in args[1:]:
            if value != res[i]:
                return False

    return True
```

1.4. Gestión de la memoria

Es de destacar que este código puede provocar un alto consumo de memoria debido a que los datos, en este caso listas o arrays de números que pueden ser extensos, son almacenados en memoria. Esto difiere de las anteriores prácticas, donde los datos eran generados sobre la marcha o el procesamiento recaía en unas pocas variables que acumulaban cuentas. De igual forma, es de interés recordar que estos datos, que se generan en el proceso maestro, son después heredados por los procesos hijos cuando se hace la operación *'fork'*, realizando así copias completas del conjunto de datos total.

No obstante y aún considerando esto, el programa troceará el conjunto inicial y pasará a cada proceso su segmento a procesar. Esto se hace así al ser el comportamiento correcto por diseño para paralelizar el problema. Esto se fundamenta en el hecho de que es posible que el programa se ejecute en un entorno de paralelización con varias máquinas (e.g., clúster). En estos entornos los procesos que ejecutan las tareas podrían no haber sido creados a partir del proceso maestro, sino de otro proceso que instancia nuestro programa, y que por ello pueden no haber 'heredado' los datos.

Puedes ver varias funciones usadas para mostrar información sobre la memoria consumida por el propio programa, así como la consumida en alojar los datos, en las siguientes líneas:

Función para indicar el consumo de memoria RAM (codigo1.py)

```
144 def read_memory_usage():
145     time.sleep(2)  # Allow time for any previous garballe collection or deletion to take place
146     total_memory, used_memory, free_memory = map(
147         int, os.popen('free -t -m').readlines()[-1].split()[1:])
148
149     process = psutil.Process(os.getpid())
150     print("RAM memory used: {0} MiB ({1}%)".format(
151         to_mebibytes_int(process.memory_info().rss), round((used_memory / total_memory) * 100, 2))
```

1.5. Tareas para la comprensión del código

Tarea 1 – Estudio previo del código

Observa en el *código1*, la función *'experiment'*, donde se han usado las funciones y el código anteriormente explicado para ejecutar los experimentos donde se usan listas y vectores, tanto con ejecución secuencial como paralela.

Tarea 2 – Ejecución del código, listas y vectores, secuencial y paralelo

Ejecuta ahora el *código1* en local y observa la salida, que muestra los tiempos en milisegundos de ejecución del procesamiento para cada tipo de datos y configuración del paralelismo. Si tu equipo tiene instrucciones vectoriales deberías observar unos resultados similares a los mostrados en la Figura 1.

- **¿Hay alguna diferencia entre el uso de listas y operaciones *clásicas*, y el uso de vectores e instrucciones vectoriales?**
- **¿Hay algún beneficio al usar paralelismo? ¿Siempre se produce este beneficio?**
- **¿Donde está tu *límite de paralelización*?**
- **¿Cómo crecen los tiempos a medida que aumenta el tamaño de los datos (a partir de 1e4)?**

Como resultado de la ejecución se debería de crear una carpeta llamada *'results_codigo1'* donde se guardarán barplots de los tiempos. Observa estos plots.

```
Removing original arrays
RAM memory used: 136 MiB (7.88%)
          1e3  1e4  1e5
SEQ LIST    6   47  631
2x LIST    19   44  359
4x LIST    23   36  214
8x LIST    29   45  191
16x LIST   55   63  167
SEQ VEC     8   52  563
2x VEC     19   43  303
4x VEC     28   40  168
8x VEC     38   46  123
16x VEC    63   78  150
```

Figura 1: Ejecución demostrativa del codigo1

Tamaño del problema y consumo de memoria: Con la implementación del programa actual, teniendo en cuenta además la paralelización y aún contando con técnicas existentes para evitar replicación masiva de datos del SO cuando se crean procesos hijos (técnica Copy-On-Write), **NO se aconseja superar los 5e6 de tamaño de conjunto de datos**. A partir de ese umbral se aconseja como mínimo tener 16 GiB de memoria.

2. Estudio del rendimiento (Amdahl)

2.1. Medición simple

Para poder medir y cuantificar una mejora de rendimiento, se puede usar la *Ley de Amdahl*. Esta ley, o fórmula, es ampliamente usada en otros campos o escenarios, pero es también aplicable a las mejoras de los tiempos mediante paralelización, considerando que en estos escenarios el *grado de paralelización* (i.e., típicamente el número de núcleos/procesos empleados) es equiparable al *factor de mejora teórico* esperable de la parte paralela de un programa. Si empleamos la fórmula de la Ley:

$$S = \frac{1}{((1-p) + \frac{p}{n})}$$

donde:

- $S \rightarrow$ Speedup, o aceleración
- $p \rightarrow$ fracción del programa que SI es paralelizable
- $1-p \rightarrow$ fracción del programa que NO es paralelizable (i.e., es siempre secuencial)
- $n \rightarrow$ número de núcleos/procesos empleados (*grado de paralelización*, o *factor de mejora teórico*)

Hay que destacar que esta fórmula tiene en cuenta que un programa a acelerar, a paralelizar en nuestro escenario, puede tener una parte del mismo que **NO es acelerable**, en nuestro caso, una parte que **NO es paralelizable** y que por ello siempre será secuencial. Esto es sumamente importante ya que en un escenario real, esta fracción es muy común y no se puede despreciar (e.g., comunicaciones, entrada/salida de datos).

En todo caso, en un escenario ideal de paralelización perfecta del programa al completo ($p = 1$), tenemos que la ley de Amdahl sería:

$$S = \frac{1}{(1-1) + \frac{1}{n}} = n$$

Es decir, que para programas paralelizables al 100 %, la aceleración equivale al número de procesadores.

En general, la ley de Amdahl nos es útil porque nos permite calcular cual debería ser el tiempo de ejecución de un programa que hemos paralelizado. Es de destacar que este cálculo puede no ser exacto por la dificultad de saber cuantificar de forma precisa el peso de las partes secuenciales ($1-p$) y paralelizables (p), y además será siempre un tiempo **'ideal'** y no **'real'**. Para obtener esta predicción, podemos usar la siguientes fórmulas:

$$Tp \approx (1-p) * Ts + \frac{p}{n} * Ts$$

$$\frac{Ts}{Tp} \approx \frac{1}{(1-p) + \frac{p}{n}}$$

donde:

- $Tp \rightarrow$ Tiempo paralelo
- $Ts \rightarrow$ Tiempo secuencial
- $p \rightarrow$ fracción del programa paralelizable
- $n \rightarrow$ Aceleración o factor ideal de mejora

No obstante, en esta última fórmula hay que destacar que estamos igualando de forma aproximada el resultado de la división de Tiempos (Ts y Tp), que son mediciones potencialmente experimentales y reales, con el resultado de aplicar la fracción de programa que creemos que es paralelo (p) y el número de procesadores (n), que arroja una aceleración ideal.

```
Removing original arrays
RAM memory used: 136 MiB (7.88%)
          1e3  1e4  1e5
SEQ LIST    6   47  631
2x LIST    19   44  359
4x LIST    23   36  214
8x LIST    29   45  191
16x LIST   55   63  167
SEQ VEC     8   52  563
2x VEC     19   43  303
4x VEC     28   40  168
8x VEC     38   46  123
16x VEC    63   78  150
```

Figura 2: Ejecución demostrativa del codigo1 *(indicando mediciones a usar en Ley de Amdahl)*

Tarea 3 – Calculando aceleración con Amdahl (local)

Del resultado de la ejecución anterior en la Tarea 2, escoge una columna (tamaño de datos) y de entre las filas, para cada tipo de datos (lista y vector), escoge su ejecución secuencial y una ejecución en paralelización. Con esos datos, calcula la aceleración obtenida para cada tipo de datos y contesta las siguientes preguntas:

- **¿Es igual esa aceleración (real) al grado de paralelismo (aceleración teórica en programa 100 % paralelo)?**
- **¿Es mayor o menor? ¿A que crees que se debe la diferencia?**
- **¿Qué tipo de datos consigue mejor aceleración real, listas o vectores?**

Tarea 4 – Calculando aceleración con Amdahl (remoto)

Repite ahora la ejecución, pero en la Máquina Remota (MR), mediante el script *'codigo1.sh'* que encontrarás en la carpeta *'run_remote'*. Comenta la sección previa que ejecutaste en local, y descomenta la sección con la configuración específica para la MR (más agresiva).

Una vez finalizada la ejecución, tráete los resultados con el script *'codigo1.sh'* de la carpeta *'retrieve'*.

Ejecución remota

```
$ bash run_remote/codigo1.sh
$ bash retrieve/codigo1.sh
```

- **¿Ves alguna diferencia con tu ejecución en local?**
- **¿Cuál es el límite de paralelismo ahora?**
- **Repite los cálculos y contesta las mismas preguntas que en la tarea anterior.**

Entrada/salida paralela y Big Data: El éxito de los nuevos paradigmas de ejecución Big Data se debe en parte a que por diseño, sus implementaciones permiten paralelizar partes de los programas que antes eran secuenciales, extendiendo además ese paralelismo a los recursos subyacentes (e.g., discos), acelerando así los programas en la parte paralela 'clásica', pero también en las partes antes secuenciales. En resumen, estos nuevos paradigmas buscan acercarse a un escenario ideal donde el procesamiento sea completamente paralelizable.

2.2. Medición avanzada

Con el anterior código (*codigo1*) las mediciones están bastante limitadas, ya que en esencia solo se están mostrando los tiempos de ejecución de cada método en su parte paralela, donde se realizan los cálculos, y además en un entorno de ejecución donde se mezclan ejecuciones de diferentes experimentos.

A fin de obtener unas mediciones más realistas y para poder cuantificar el efecto de la parte secuencial del programa (generación de los datos), usaremos el *código2*. Este código realiza las mismas operaciones y los mismos cálculos que el *código1*, pero solo realiza 1 experimento por cada ejecución del programa, delegando la configuración de dicho experimento a las variables que le pasaremos al programa por la línea de comandos. Un script bash será el encargado ahora de realizar los diversos experimentos mediante la ejecución con la configuración adecuada. Con este diseño de ejecución experimental, no tendremos la tabla final de resultados del *código1*, pero sí que tendremos unas mediciones mucho más precisas.

No obstante, es importante destacar que en este código se ha 'exagerado' la parte secuencial mediante una opción de uso de un fichero externo donde se alojan los datos a procesar, y una lectura artificialmente más lenta de dicho fichero. De esta forma y a modo de experimentación, el *codigo2* permitirá tanto una ejecución con datos leídos de un fichero (parte secuencial lenta) como una ejecución con datos generados internamente (parte secuencial rápida), el modo ya anteriormente usado en el *código1*.

Tarea 5 – Evolución de las aceleraciones, medición en la MR

Ejecuta el script *'codigo2.sh'* con el parámetro 'amdahl' para realizar los diferentes experimentos que nos permitirán re-evaluar las aceleraciones usando esta Ley. Este script mandará la ejecución a la MR. A diferencia del código anterior, este no generará un resultado por la salida estándar, sino que generará ficheros '.txt' con la salida de las ejecuciones, así como información recabada por el comando *time*. Estos ficheros se guardarán en la MR en la carpeta *'results_amdahl'*, una vez finalizado el trabajo podrás traerlos con el script *'codigo2.sh'* de la carpeta *'retrieve'* y el argumento *'amdahl'*. Este script también generará una salida de datos que usaremos a continuación.

```
$ bash run_remote/codigo2.sh amdahl
$ bash retrieve/codigo2.sh amdahl
```

- **Ejecuta el codigo2 y los experimentos de Amdahl en la MR y recupera los ficheros resultados**
- **Observa la salida de alguno de estos ficheros**

```
$ cat results/amdahl/17par_8_list_file.txt
```

```
alumno@ParalelismoMV:~/Compartida/P4-InstruccionesSIMD$ cat results/amdahl/13par_8_vec_file.txt
N_MAX is 1e6
MODULE is  12500.0
Running in parallel with a degree of 8x
Data will come from file
Running vectors
Max size of lists or arrays is 1e6
Reading data from file
Generation of the data took 8.5638 seconds
Size of ax is:   7 MiB
Size of bx is:   7 MiB
Computation took 0.9303 seconds
Program execution took 9.4942 seconds
DATA|par_8_vec_file;8.56;0.93;9.49
        Command being timed: "python3 codigo2.py vec par 8 1e6 file"
        User time (seconds): 9.23
        System time (seconds): 0.88
        Percent of CPU this job got: 105%
        Elapsed (wall clock) time (h:mm:ss or m:ss): 0:09.61
        Average shared text size (kbytes): 0
        Average unshared data size (kbytes): 0
```

Figura 3: Salida de un fichero de la ejecucion del codigo2 para experimentos de Amdahl

La salida de este código y de los scripts la usaremos para analizar más en detalle las mejoras obtenidas, frente a las mejoras teóricas. Para ellos usaremos una hoja de cálculo que tienes disponible en Moodle con el nombre de *'analisisP4.ods'*. Esta hoja de cálculo contiene las fórmulas que se pueden usar para medir la aceleración obtenida y lo cerca que está de la teórica. Para usar esta hoja será necesario pegar los datos que obtendremos más adelante en ella, con cuidado de no sobreescribir las celdas de fórmulas, y extendiendo dichas celdas si se desease.

Tiempos de ejecución: Las ejecuciones que haremos en esta práctica pueden tener unos tiempos considerablemente largos, del orden de minutos o incluso de hasta 15 minutos en el caso de las ejecuciones que se pedirán a continuación. Se recomienda tener paciencia y observar la salida del programa para intuir su progreso y trabajo pendiente.

En caso de realizar la práctica fuera de clase, se recomienda consultar el horario de disponibilidad de la MR y adecuarse a él.

Tarea 6 – Estudio de la escalabilidad con Ley de Amdahl y ejecuciones previas

Abre la hoja de cálculo *'analisisP4'* y observa la hoja llamada 'Amdahl'. Re-ejecuta ahora el script *'codigo2.sh'* de la carpeta *'retrieve'* para obtener los datos a pegar en esta hoja. Busca la celda de color azul claro y pega los datos, al hacerlo se debería auto-detectar que están en formato CSV y se debería de abrir un menú. Rellena a mano la columna de 'N cores'.

Al copiar los datos, deberían calcularse automáticamente varios datos:

- La parte secuencial $(1 - p)$ y la parte paralelizable (p).
- A partir de los datos anteriores, debería calcularse la aceleración teórica para cada experimento (S).
- A partir de los tiempos, se calculan las aceleraciones reales de la parte de procesamiento de datos, y de la ejecución en global.
- Se calculan el grado en el que se han acercado las aceleraciones reales a la teórica (eficiencia).
- Se calcula el overhead de recursos, es decir, cuantos recursos (tiempo de CPU) más han usado las versiones paralelas respecto a la secuencial, teniendo en cuenta que la carga de trabajo es la misma.

Analiza los resultados:

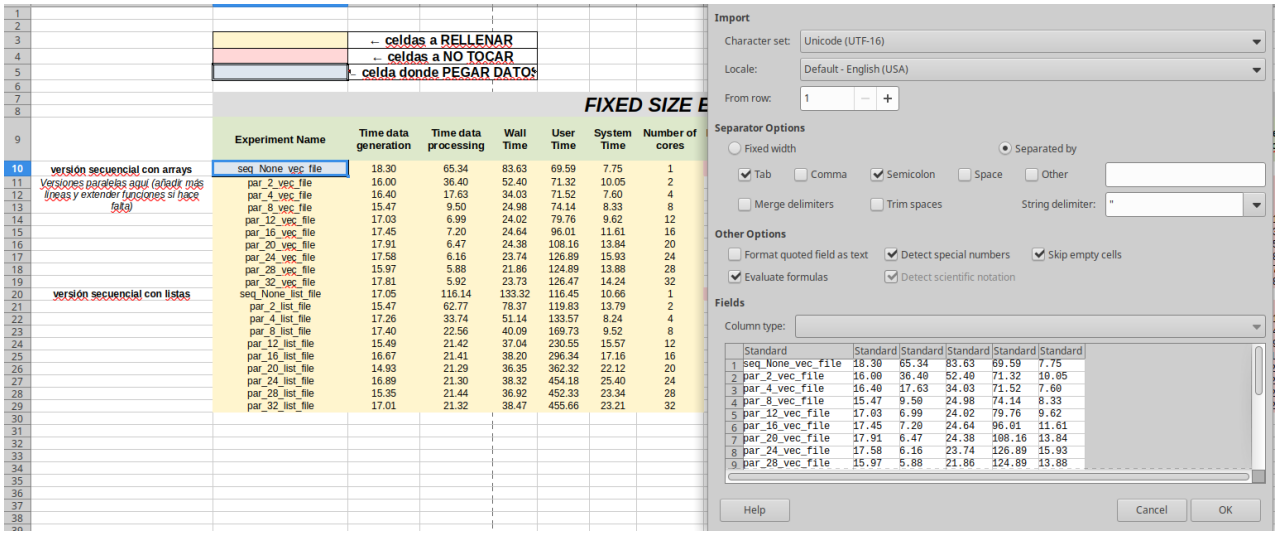
- **¿Se acerca la aceleración de cálculo (*Speedup processing*) a la aceleración ideal, es decir al grado de paralelismo (*Number of cores*)?**
- **¿Se acerca la aceleración del programa en conjunto (*Speedup global*) a la aceleración teórica según la ley de Amdahl para ese programa (*Max speedup*), con esa fracción de secuencial y paralelo?**
- **¿Hay diferencias significativas en las aceleraciones entre los experimentos que procesan los datos usando listas y los que usan vectorización?, ¿Y en las eficiencias?**
- **¿Cómo evoluciona el overhead de recursos a medida que se incrementa el paralelismo?**
- **¿Qué tipo de estructura de datos (lista o vectores) tiene mejor comportamiento en lo que respecta a este overhead?**

Código de generación de datos: Para la generación de los datos se ha usado un código Python llamado 'generate_data.py'. Este código recibe un parámetro que será la potencia de 10 a usar para generar los datos. El resultado de su ejecución serán 4 ficheros de mismo tamaño y con mismo contenido, pero que serán usados a fin de crear los conjuntos de datos en forma de listas o de arrays en las ejecuciones del código2. Puedes ver este comportamiento en las siguientes líneas del programa:

Programa para generar datos (generate_data.py)

```
31    print("Generating numbers")
32    numbers = [*range(1, N_MAX + 1)]
33    with open('a.txt', 'w') as filehandle:
34        filehandle.writelines("%d\n" % n for n in numbers)
35
36    copyfile('a.txt', 'ax.txt')
37    copyfile('a.txt', 'b.txt')
38    copyfile('a.txt', 'bx.txt')
```

Copia de datos en formato CSV en LibreOffice: LibreOffice detecta cuando se están copiando datos en un formato potencialmente tabular (CSV), y permite en una ventana nueva elegir la configurcion de parseo de dichos datos, así como ver una previsualización, antes de finalmente pegar los datos en la hoja. **En este caso se usa el semicolon (';') como separador.**



Formato de números decimales: El uso y comportamiento de los programas de ofimático de tipo 'hoja de cálculo' depende de variables locales a cada ordenador. En este caso estas diferencias pueden afectar a cómo se interpretan los **números decimales**.

- En el formato inglés, el punto ('.') es separador
- En el formato no inglés, la coma (',') es separador

Por defecto, los códigos usan el **punto** ('.') como separador decimal. Si ves algún tipo de error en las fórmulas, comprueba si se debe a que el formato esperado y al de los datos es distinto. Si este es el caso, modifica la variable **DECIMAL_SEP** que encontraras en el script *'retrieve/codigo2.sh'*.

3. Estudio de la Escalabilidad (Gustafson)

No obstante hay que destacar que la Ley de Amdahl tiene un punto débil, y es que considera que la carga de trabajo ejecutada por los programas es siempre la misma, dejando que sea la aceleración la que varíe.

Esto provoca que con una misma carga de trabajo, si esta no es lo suficientemente grande, tarde o temprano se alcanzará un punto en el que añadir más recursos no arrojará las mismas ganancias. Este escenario es precisamente el explorado en las tareas anteriores. El problema es que esto nos impide saber si el programa escala realmente, ya que aunque sí que se observe una mejoría a medida que se añaden recursos, no sabemos si esa mejora se mantiene si se aumenta de forma proporcional el tamaño de los datos.

Idealmente, nuestros programas paralelos deberán escalar tanto si se aumentan los recursos y se mantiene el tamaño de los datos fijo, algo denominado *escalabilidad fuerte* (al ser más difícil), como si se aumenta el número de recursos y de forma proporcional, el tamaño de los datos, algo que es llamado la *escalabilidad débil* (al ser unas condiciones más relajadas). Este último concepto es también referido como la *Ley de Gustafson*.

Esta Ley nace de la observación de ejecuciones reales en entornos de supercomputación, y su idea nuclear es que **en un escenario de ejecución paralela, ante un aumento de los recursos dado, los programas tenderán a realizar ejecuciones con más datos, o ejecuciones más intensivas, todo con el objetivo de aprovechar esos recursos**. Esto es algo muy esperable en experimentos científicos, donde ante un aumento de las capacidades, se suele preferir mantener los tiempos de experimentación a cambio de obtener unos resultados más fiables o precisos.

Tarea 7 – Estudio de la escalabilidad con Ley de Gustafson y ejecuciones previas

Ejecuta ahora los experimentos para medir la escalabilidad cuando el tamaño de los datos crece de forma proporcional a los recursos usados. Ejecuta el script *'run_remote/codigo2.sh'* con el argumento *'gustafson'*. Esto mandará la tarea a la MR. Una vez finalizada la ejecución, tráete los resultados con el script *'retrieve/codigo2.sh'* y el argumento *'gustafson'*.

```
$ bash run_remote/codigo2.sh gustafson
$ bash retrieve/codigo2.sh gustafson
```

Copia los datos en la misma hoja de cálculo anterior, pero ahora en la página 'Gustafson'.

- Teniendo en cuenta que ahora los datos provienen de una generación interna del programa y que por ello los tiempos de generación son próximos a 0, **¿cómo evolucionan los tiempos globales en este caso? ¿Es esto lo normal o esperable?**
- **¿Cómo son las pendientes (diferencial respecto al secuencial, $dtiempo/dncores$) a medida que aumenta el grado de paralelismo?, teniendo en cuenta que la pendiente ideal es 0 (el tiempo se mantiene).**
- **¿Observas algún punto a partir del cual la MR comienza a tener un peor rendimiento? ¿Podrías explicarlo?**

Copyright & Disclaimer


```
# Copyright © 2025 by Jonatan Enes (jonatan.enes@udc.es)
# Computer Engineering department, Universidade da Coruña, Spain.
#
# This file is part of several courses on parallel processing from Universidade da Coruña,
# and it can only be used and/or modified by the author, the students or any
# explicitly authorized person by the author, inside the context of the subject.
# Redistribution to a third-party without previous authorization is strictly forbidden.
# No commercial use is allowed.
#
# This file has only academic purposes and should not be used for any real-world
# scenario, the author holds no accountability for its use or misuse.
```